

Multi-Agent Systems



4 Nov 2025 – Lecture 3 – Prolog Datatypes

Data in prolog

- Relational representation instead of direct value manipulation.
 - Facts, rules and queries instead of functions.
- Unification instead of assignment.
- Terms instead of datatypes with fixed definitions.

Basic types of terms:

Atoms, numbers, variables, lists, and compound terms.

(Strings)

Prolog - atoms

Multi-Agent Systems

Lecture 3, part 1

Atoms

The basic building blocks of prolog.

An **atom** is one of the following:

1. A sequence of letters, digits, or underscores that *starts with a lower-case letter*, e.g. car, hOUSE_2, father.
2. A sequence of letters, digits or underscores that are *enclosed in single quotes*, e.g. 'car', 'House', 'HOUSE 2'.
3. A string of special characters, e.g. :- or ,

An atom is a constant and a term.

Prolog – Compound terms

including negation

Multi-Agent Systems

Lecture 3, part 2

Compound term

functor(arguments) $\rightarrow f(t_1, \dots, t_k)$

- # arguments = arity k
- functor/ k is the signature
- A term is ground when it contains no (uninstantiated) variables.
- Compound term $f(t_1, \dots, t_k)$ is a valid term if each t_i is a term and f is an atom.

For example:

- $\text{parent}(\text{john}, \text{mary})$ $\square$ signature: $\text{parent}/2$.
- $f(g(a), h(b))$ $\square$ ground.
- $q(Z)$. $\square$ Not ground.
- $Z=a, q(Z)$ $\square$ ground

Negation \+

Prolog allows a predicate `\+ /1` which models a specific type of negation. In short, it means:

`\+ f` is true in program `P` iff `f` cannot be derived from `P` and the program terminates.

Compare the difference(!):

- $\neg p$ in logic: proposition `p` is false
- `\+ p` in Prolog: there is not enough information to derive `p`.

`not(...)` works the same as `\+`, but should no longer be used.

Definition

A search tree for a program P for a goal G is called **finitely failed** if there is:

- **no successful branch** in the tree and
- **also no infinite branch.**

Negation as finite failure

The operator $\backslash +$ is called **negation with finite failure**.

$\backslash + G$ can be derived from P iff the search tree of P for G is finitely failed.

Quiz

Consider the program and queries:

```
daughterOf(alice,bob).  
daughterOf(jessica,mary).  
sonOf(paul,mary).
```

```
onlySisters1(X):- \+ sonOf( _, Z), daughterOf(X,Z).  
onlySisters2(X):- daughterOf(X,Z), \+ sonOf( _, Z).
```

```
?- onlySisters1(A).  
?- onlySisters2(A).
```

We would like to derive every person that is an only daughter.

Which of these queries will return the right answer 'alice'?

1: ?- onlySisters1(A).

2: ?- onlySisters2(A).

Prolog – Numbers

including arithmetic

Multi-Agent Systems

Lecture 3, part 3

Numbers

Integers: Whole numbers (e.g., 7, -3).

Floating-point numbers: Decimal values (e.g., 3.14, -0.5).

Built-in arithmetic functors

Examples of important ones:

- $+/2$ addition
- $-/2$ subtraction
- $-/1$ negative number
- $*/2$ multiplication
- $//2$ (floating point) division
- $///2$ (integer) division
- $\text{mod}/2$ remainder after division
- $\text{max}/2$ maximum of two numbers

BUT: Terms are NOT evaluated

?- 4 + 3 = 7.

false.

Operators that force evaluation

- $=:=$ /2 (left *and* right evaluation of terms)

?- $4 + 3 =:= 5 + 2$.

true.

- $= \backslash =$ /2 (inequality)

?- $4 + 3 = \backslash = 5 + 2$.

false.

Comparison Operators

Force left and right evaluation.

- $<$ /2 (strictly larger)

?- $4 + 2 < 5 + 3$.

true.

Other comparison operators:

- $=<$ /2 (larger or equal; not $<=$!)
- $>$ /2 (strictly smaller)
- $>=$ /2 (smaller or equal)

Evaluating terms with variables

?- $4 + 3 ::= X + 2$.

ERROR: $::=/2$: Arguments are not sufficiently instantiated

?- $4 + 3 =\backslash X + 2$.

ERROR: $=\backslash/2$: Arguments are not sufficiently instantiated

?- $Y < 5 + 2$.

ERROR: $</2$: Arguments are not sufficiently instantiated

?- $Y = < 5 + Z$.

ERROR: $= </2$: Arguments are not sufficiently instantiated

Special is operator

- The is operator takes an arithmetic expression on its **right side**, evaluates it, and *unifies* the result with the variable on its **left side**.
- **Left side:**
 - Variable or number
 - it checks if this value matches the evaluated result of the expression on the right.
- **Only for arithmetic expressions**

?- 4 is 2 + 2.

true.

?- 2 + 2 is 2 + 2.

false.

?- 2 + 2 is 4.

false.

?- X is 2 + 2.

X=4.

Special **is** operator

?- 4 is X + 2.

ERROR: is/2: Arguments are not sufficiently instantiated

?- X = 2, 4 is X + 2.

true.

Exercise

Query:

Output:

?- $2 + 2 = 2 + 2$.

?- $4 = 2 + 2$.

?- $X = 2 + 2$.

?- $4 \text{ is } 2 + 2$.

?- $2 + 2 \text{ is } 4$.

?- $X =:= 2 + 2$.

?- $3 + 2 =:= 2 + 3$.

Exercise

Query:

?- $2 + 2 = 2 + 2$.

?- $4 = 2 + 2$.

?- $X = 2 + 2$.

?- $4 \text{ is } 2 + 2$.

?- $2 + 2 \text{ is } 4$.

?- $X ::= 2 + 2$.

?- $3 + 2 ::= 2 + 3$.

Output:

true.

false.

$X = 2 + 2$.

true.

false.

not sufficiently instantiated

true.

Some other operators

The remainder of 7 divided by 2 is 1:

1 is $7 \bmod 2$.

The value of 2^{10} is 1024:

1024 is 2^{**10} .

Compute maximum of 4 and 89:

X is $\max(4, 89)$.

Example

Define the predicate `addThreeAndDouble/2` which should compute the function:

$$f(x) = 2(x + 3)$$

Exercise

Define the predicate `addThreeAndDouble/2` which should compute the function:

$$f(x) = 2(x + 3)$$

`addThreeAndDouble(X,Y) :- Y is 2*(X+3).`

X must be instantiated!

Prolog – Lists

Multi-Agent Systems

Lecture 3, part 4

Lists are an important data structure in Prolog. They can store (compound) terms.

The following are examples of lists:

- [michelangelo, donatello, raphael, leonardo]
- [michelangelo, ninja(donatello), X, 2, michelangelo]
- []
- [michelangelo, [donatello, raphael], [splinter, sensei(splinter)]]
- [[], mutant(z), [2, [b,c]], [], Z, [2, [b,c]]]

$[\]/2$ operator

Internally, list are defined recursively using a functor $[\]/2$ which allows to “concatenate” terms:

- $[\]$: the empty list
- $[\](a, [\])$: corresponds to $[a]$.
- $[\](a, [\](b, [\]))$: corresponds to $[a,b]$.
- $[\](a, [\](b, [\](c, [\])))$: corresponds to $[a,b,c]$.

Definition

- $[t_1, t_2, \dots, t_k]$ denotes a list of k elements.
- with **head** t_1 and **tail** $t_2 - t_k$.
- **[Head | Tail]**

Note that:

- the **head** of a list is a **term**,
- the **tail** of a list is a **list** itself.

Empty list:

- $[]$ is the empty list
- empty list has neither tail nor head

Examples of lists

1. $[1 \mid [2, 3, 4]]$
= $[1, 2, 3, 4]$.

2. $[1, 2, 3 \mid []]$
= $[1, 2, 3]$.

3. $[1 \mid 2, 3, 4]$
□ not a list

4. $[[] \mid []]$
= $[[]]$

5. $[[1, 2], [3, 4] \mid [5, 6, 7]]$
= $[[1, 2], [3, 4], 5, 6, 7]$

Use of head | tail structure

1. $Y = [1, 2, 3, 4]$, $[A, X | B] = Y$.

$X = ?$

2. $Y = [1, 2, 3, 4]$, $X = [5 | Y]$.

$X = ?$

3. $Y = [1, 2, 3, 4]$, $[A | X] = Y$.

$X = ?$

Use of head | tail structure

1. $Y = [1, 2, 3, 4]$, $[A, X | B] = Y$.

$X = 2$

2. $Y = [1, 2, 3, 4]$, $X = [5 | Y]$.

$X = [5, 1, 2, 3, 4]$

3. $Y = [1, 2, 3, 4]$, $[A | X] = Y$.

$X = [2, 3, 4]$

List built-in predicates

```
?- member(a, [a,b,c]).  
true .
```

```
?- append([a,b,c], [x, y, z], AppendedList).  
AppendedList = [a, b, c, x, y, z].
```

```
?- nth0(2, [a,b,c,d], E).  
E = c.
```


Prolog – Strings

Multi-Agent Systems

Lecture 3, part 5

Four ways to represent a string

In SWI-Prolog

1. **Atoms:** typically used for identifiers and comparison and not for manipulation.
 - `car(ferrari).`
 - `car('Ferrari 430 Scuderia').`
2. **Lists of characters:**
 - `"hello"` $\square$ `[h, e, l, l, o].`
3. **Lists of character codes:** typically used when parsing.
 - `"hello"` $\square$ `[104, 101, 108, 108, 111].`
 - `Read_line_to_code/2` is a built-in text reading predicate
4. **String type**

String type in SWI-Prolog

- Represent text as a unit but can be composed, decomposed or converted.
- Double quotes indicate a string type

string_concat/3.

```
?- string_concat("Hello", " world", S).  
S = "Hello world".
```

sub_string/5.

```
?- sub_string("Hello World", _, _, _, "Hello").  
true.  
?- sub_string("Hello World", 0, 1, _, Out).  
Out = "H".
```

Prolog – Variables

*including the anonymous variable and
instantiation patterns*

Multi-Agent Systems

Lecture 3, part 3

Variables

A variable is a sequence of letters, digits or underscores that starts with an upper-case letter or an underscore, e.g. X, Car, HOUSE_2, _father, House.

- The variable _ is called the anonymous variable.

Example (Anonymous variable)



?- [X1,X2,X3,X4 | Tail] = [[], d(z),[2,[b,c]],[],Z].

X1 = X4, X4 = [], X2 = d(z), X3 = [2, [b, c]], Tail = [Z]

Only interested in 2nd and 4th component:

?- [_, X2, _, X4 | _] = [[], d(z),[2,[b,c]],[],Z].

X2 = d(z), X4 = []

Compare with:

?- [Y,X2,Y,X4 | Y] = [[], d(z),[2,[b,c]],[],Z].

false

Use of head | tail structure (revised)



1. $Y = [1, 2, 3, 4]$, $[_ , X | _] = Y$.

$X = ?$

2. $Y = [1, 2, 3, 4]$, $X = [5 | Y]$.

$X = ?$

3. $Y = [1, 2, 3, 4]$, $[_ | X] = Y$.

$X = ?$

Instantiation patterns

Definition:

`addThreeAndDouble(X,Y):- Y is 2*(X+3).`

`?- addThreeAndDouble(X,10).`

ERROR: is/2: Arguments are not sufficiently instantiated

Variable X must be instantiated when predicate is queried!

Instantiation patterns

What is the instantiation pattern?

`% addThreeAndDouble(+X, ?Y).`

`addThreeAndDouble(X,Y):- Y is 2*(X+3).`

`+X` : variable X is input variable and must be instantiated when predicate is queried.

`?Y`: variable Y does not have to be instantiated.

Planning:

- Thursday November 6
 - Practical: work on prolog assignment 1
 - **No lecture**
- Monday November 10
 - **Deadline assignment 1**
- Tuesday November 11
 - Lecture: Recursion I
 - Preparation: Learn Prolog Now Ch. 3, 4.4